SoK: The Weakest-Link Principle in Public Key Infrastructures and Modern Mitigation Strategies

Kertis Mwanza and Carsten Köhn
Department of Electrical Engineering and Computer Science
Bochum University of Applied Sciences
Bochum, Germany
Email: kertis.mwanza@stud.hs-bochum.de, carsten.koehn@hs-bochum.de

***Abstract*—As digital transformation accelerates, securing communication through hierarchical Public Key Infrastructures (PKIs) is increasingly critical. Yet, this centralized trust architecture remains inherently vulnerable. As a Systematization of Knowledge (SoK), this paper maps the threat landscape of hierarchical PKIs, demonstrating how a compromise at any single node from a Root CA breach to an operational revocation failure can trigger a cascading loss of global trust. Grounded in the Weakest-Link Principle, our analysis reveals that a PKI ecosystem is only as resilient as its least protected vector. Traditional revocation mechanisms, particularly CRLs and OCSP, exhibit significant operational and privacy flaws, and are often rendered ineffective by client-side "soft-fail" policies. To address these vulnerabilities, we advocate for a shift: replacing unconditional trust in individual entities with decentralized,verifiable protocols. We evaluate Certificate Transparency (CT) as a core mitigation strategy, illustrating how append-only Merkle trees make misissuance publicly visible and cryptographically auditable. Finally, we synthesize essential operational hardening measures such as strict key cryptoperiods and procedural policies to ensure long-term ecosystem resilience.**

***Index Terms*—Public Key Infrastructure, Certificate Transparency, Vulnerability Analysis, Weakest-Link Principle, Applied Cryptography, OCSP, OCSP-Stapling, Revocation**

I. Introduction

The ongoing digital transformation is permeating every aspect of the economy and daily life. To continue ensuring the safety of citizens, critical infrastructure and business under these circumstances, a secure digital infrastructure is essential. A recent study by Bitkom underscores the severity of the threat: Nearly three-quarters of all companies surveyed report feeling highly threatened by cyberattacks [1, p. 2].

However, this digital transformation not only opens up socio-economic opportunities but also creates new vulnerabilities for the “cybercriminal shadow economy” [2, p. 19]. A fundamental pillar of this digital infrastructure is the assurance of authenticated identities and confidential communication. This is technically achieved through a Public Key Infrastructure (PKI). A PKI is a hierarchical system comprising hardware, software, and policies that manages digital certificates to make confidentiality and integrity technically enforceable in an insecure network environment.

Despite its role as a security anchor, this centralized architecture is not invulnerable. Historical and current security incidents show that compromises of Certificate Authorities CAs or weaknesses in validation processes can shake global trust in a PKI. This tension gives rise to two central research questions for this paper:

1) What attack vectors and vulnerabilities can undermine trust in a hierarchical public-key infrastructure?
2) How can these threats be systematically classified and their risk potential minimized?

This paper argues that the security of the PKI ecosystem cannot rely solely on cryptographic algorithms. Rather, the system requires a paradigm shift from unconditional trust in individual entities to decentralized, mathematically verifiable procedures and strict procedural hardening measures.

To support this argument, section II introduces the cryptographic primitives and section III the fundamental properties of a PKI. Building upon these concepts, section IV systematically classifies attack vectors and evaluates them using a real-world security incident. These threats are subsequently contrasted with modern hardening mechanisms in section V. Finally, section VI synthesizes the findings of this paper and provides an outlook on the future of the Web PKI ecosystem.

II. Security Objectives and Cryptographic Primitives

The security of communication in open network environments is based primarily on the enforcement of the classic security objectives of information security: confidentiality, integrity, and authenticity.

This section explains the fundamentals needed to understand the scope of the attack vectors analyzed in section IV. It begins by outlining the mathematical foundations, with a specific focus on asymmetric cryptography, hashes, and digital signatures, thereby establishing the technical basis for the security objectives. At the center of every PKI is asymmetric cryptography (public-key cryptography). Asymmetric cryptosystems use two mathematically related keys (a key pair). To ensure this mathematical relationship, such cryptosystems rely on the use of so-called trapdoor one-way functions.

Mathematically speaking, a function $f : X \to Y$ is such a function if, as mentioned in [3, Ch. 1.1], it satisfies the following properties:

1) The computation of the function value $y = f(x)$ is efficient for every $x \in X$.
2) The computation of a preimage $x = f^{-1}(y)$ is computationally inefficient for a given y.
3) With knowledge of the additional information, f can be efficiently inverted.

In cryptography, this additional information is referred to as the private key (k_{priv}). Only those in possession of this information can efficiently invert f. The corresponding public key (k_{pub}) allows only the forward direction. Together, they form a key pair.

Today's common implementations are based on number-theoretic problems whose solution requires enormous resources (computing power and/or time) without the private key. [4] lists the following:

- The factorization problem (e.g., RSA): The difficulty of decomposing a large composite number into its prime factors.
- The discrete logarithm problem (e.g., Diffie-Hellman, ECC): The difficulty of determining the exponent in cyclic groups (or on elliptic curves)

In PKI, methods based on such asymmetric operations are primarily used to achieve ensures that data is not manipulated (altered, deleted, or replaced) without authorization

Technically, this is ensured by digital signatures. A digital signature is the cryptographic equivalent of a handwritten signature, but offers stronger evidence of non-repudiation [5, p. 23].

Adapting the principles from [4, Chapter 10.1.2], a digital signature scheme can be formalized as a tuple (Sig, ver), where:

1) (k_{priv}, k_{pub}) represents a key pair.
2) Sig is a signing algorithm that generates a signature $s = \text{Sig}(x)$ using the private key k_{priv} and a message x.
3) ver is a verification algorithm that checks whether the signature s corresponds to the message x and the public key k_{pub}. Formally, this yields:

$$\text{ver} : (x, s, k_{pub}) \to \{\text{true}, \text{false}\} \tag{1}$$

In Praxis, however, Sig is rarely applied to an entire message x, but rather, for reasons of efficiency and security, to its hash value [4, p.336]. A hash value (often simply called a hash) is, in turn, the result of a cryptographic function H that maps a string of arbitrary length (input) to a string of fixed length (output) [4, p. 345]. Mathematically, this results in the following function:

$$H : \{0,1\}^* \to \{0,1\}^n \tag{2}$$

Where: $\{0,1\}^*$ is the message space of arbitrary length, and $\{0,1\}^n$ is the set of all strings of fixed length n. To be secure hash functions need to fullfil following properties [4, p.339-341]:

1) **Preimage Resistance:** it is computationally impossible to find a message x such that $h(x) = z$.
2) **Collision Resistance:** It is computationally impossible to find two different messages $x_1 \neq x_2$ that produce the same hash value, i. e. $h(x_1) = h(x_2)$.
3) **Second Preimage Resistance:** Given a message x_1 and its hash value $z_1 = h(x_1)$ it is impossible to find another message $x_2 \neq x_1$ such that $h(x_2) = z_1$.

The key advantage of these properties lies in determinism. Any modification to the message necessarily results in an alternative hash value and thus an invalid signature. Recipients can therefore independently verify whether the integrity of the data was preserved during transmission.

However, these mathematical methods have a significant limitation: While they can verify that a message was signed with a specific private key, they do not reveal who actually owns that key. An attacker could generate their own key pair and impersonate an authorized communication partner; authenticity would thus not be guaranteed. To close this security gap, the components of a PKI listed in the following section are necessary.

III. Key Elements of a Public Key Infrastructure

A PKI is characterized in particular by the orchestration of its cryptographic algorithms. To close the gap between mathematical verifiability and identities highlighted in section II, this interaction is essential.

As mentioned in [5], a PKI is a framework consisting of hardware, software, personnel, procedures, and policies that serves to issue, manage, and, if necessary, revoke digital certificates. The overarching goal is to establish trust, which enables two unrelated parties to communicate securely with one another. Before the individual elements can be subjected to a thorough analysis, the concept of an entity must be clarified. The NIST fundamentally defines an entity as an individual, organization, device, or process [5, p. 10]. Applied to the context of this work, the term broadly encompasses all participants and system components that are actively involved in the PKI process or are identified by certificates.

To illustrate the functional dependencies within this ecosystem, the core components based on the reference architectures [6], [7], [8], are divided into three areas:

1) Trust Services: The part of the infrastructure that verifies identities and initiates processes to authenticate them.
2) Governance: Documents that define procedural guidelines.
3) Participants and validation: The actors who verify the validity status (of e.g., certificates) or use them.

At the center of all trust-establishing entities is the Certificate Authority (CA). It operates as the highest trust authority within the ecosystem and ensures trust between PKI participants and relying parties. It is the determining factor for the authenticity and integrity of the data in issued certificates. Its primary task is to establish the cryptographic link between an identity and its public key through its own digital signature [6, p. 11]. Since the compromise of its private key would allow attackers to issue fraudulent certificates (rogue certificates), the confidentiality of this key is of the highest priority.

To maintain the security of the system, the CA delegates the identification and authentication of certificate applicants to the Registration Authority (RA). In doing so, the RA acts as an interface between the applicant and the CA. Furthermore, it validates the submitted application data and verifies the applicant's eligibility. However, the RA does not sign certificates itself [7, p. 9], as it does not have access to the CA's private key. As a result of insufficient identity checks by the RA, attackers can obtain the issuance of rogue certificates. Therefore, the RA poses a critical security risk and always operates in accordance with the predefined certificate policy.

Furthermore, the Federal Office for Information Security [6, p. 26] mandates the use of cryptographic modules, such as hardware security modules (HSMs), for applications requiring a high level of security (particularly root CAs). Such a module is defined as a tamper-resistant hardware device for generating, storing, and using cryptographic key material. In this context, it serves as a key trust factor by storing the CA's private key in a tamper-resistant environment. In particular, standards such as FIPS 140-2/3 define strict requirements for physical tamper protection for higher security levels (Level 3/4) [9]. This block is rounded out by the repository, which stores valid, archived, and revoked certificates to make them available to entities at any time [8, p. 8]. Because entities rely on the availability and integrity of the data contained in the repository, unauthorized write access to a repository must be prevented to stop fraudulent certificates or revocation lists. Furthermore, denial-of service and replay attacks (e.g., the import of outdated revocation lists) are also critical to security.

For technical trust services to operate properly, established standards are needed to clarify responsibilities. With reference to [6, Chapter 6.1], the governance segment is divided into two sub-areas, the distinction between which is essential:

1) The Certificate Policy (CP) is described as a "designated set of rules that specifies the applicability of a certificate to a specific user group and/or application class with common security requirements" [10, p. 9]. Consequently, its purpose is to define the obligations.
2) In contrast, the Certification Practice Statements (CPS) provide detailed documentation of the practices a CA performs to comply with the respective CPs (e.g., a key ceremony or specific verification procedures).

The result of these standards is, in turn, the digital certificate. A digital certificate is a cryptographically signed file that contains details about a person or organization in the X.509 description language. Fig 1 illustrates the structure of X.509 certificates according to [8, Chapter 4.1].

The central components of the *tbsCertificate* can be summarized as follows, according to [8]:

```
Certificate  ::=  SEQUENCE  {
    tbsCertificate       TBSCertificate,
    signatureAlgorithm   AlgorithmIdentifier,
    signatureValue       BIT STRING
}
```

Figure 1: Top-Level Structure of the X.509 Certificate.

1) **Version**: Specifies the format of the certificate (today almost exclusively v3 to support extensions).
2) **Serial Number**: An integer assigned by the certificate authority (CA) that is uniquely identifiable.
3) **Signature Algorithm**: The algorithm used by the CA to sign the certificate.
4) **Issuer**: The Distinguished Name (DN) of the issuing CA.
5) **Validity**: The period (defined by *NotBefore* and *NotAfter*) during which the certificate is valid. Before or after this window, the certificate is invalid.
6) **Subject**: The DN of the entity (e. g., user or server) whose public key is being certified.
7) **Subject Public Key Info**: The owner's actual public key and the algorithm used (e. g., RSA).
8) **Extensions**: A field containing critical additional information.

To complete the X.509 structure, the CA must sign the tbsCertificate field with its digital signature by performing the following mathematical operation:

$$\text{signature} = \text{Sig}_{K_{Priv}}(\text{Hash}(\text{Version} \parallel \text{Serial} \parallel \ldots \parallel \text{Public Key} \parallel \text{Extensions})) \quad (3)$$

The resulting signature is stored in the *signatureValue* field, with the signature algorithm used specified in the *signatureAlgorithm* field. The final digital certificate is ultimately the combination of the data block and the signature. In other words:

$$\text{certificate} = \text{tbsCertificate} \parallel \text{signatureAlgorithm} \parallel \text{signature}$$

where $\parallel$ denotes the concatenation of the data.

The resulting certificate can then be used by relevant entities for validation

The usability of these certificates depends largely on the relying parties. Their role is to technically validate the digital signature, the current validity status, and the entire certification chain. Although the term "relying party" may seem abstract,

it generally refers to tangible software components or devices that use certificates. Common examples include web browsers, operating systems, IoT devices, or, in highly secure scenarios, web servers.

The primary purpose of this validation is to verify the *key binding* that is, the cryptographic link between an identity (the participant) and a public key. Since this binding is only trustworthy as long as the corresponding private key remains under the participant's sole control, the participant is obligated to keep it confidential [10, p. 4]. In the event of loss or suspected compromise, the participant must immediately notify the CA to initiate a revocation

A revocation occurs when a certificate's validity is revoked before the end of its regular validity period (NotAfter). A revocation is performed in the event of any security critical incidents. A revoked certificate may no longer be accepted by verifying entities to meet security objectives. To communicate such revocations to verifying authorities, the practice of maintaining a Certificate Revocation List (CRL) has become standard. A CRL is a time-stamped and signed list that contains information about revoked certificates. In particular, the serial numbers of the respective certificates are publicly accessible [8, p. 13].

However, the use of a CRL is inevitably associated with delays. Since these lists are typically updated periodically [11] and stored by the verifying authorities, a security-critical time window arises between revocation and the verifying authority's awareness of it.

To address this security risk, the *Online Certificate Status Protocol* (OCSP) was developed as a dynamic alternative. OCSP operates according to a synchronous query model between the client and the CA or its OCSP responder. The responder returns either a definitive status response or an error message [11, p. 5]. The purpose of this procedure is to provide a near-real-time alternative to periodic CRLs.

The analysis so far has primarily considered the functional components of a PKI in isolation. In practice, however, these rarely operate independently. If, for example, a PKI were based exclusively on a single CA, the compromise of its private key would compromise the entire trust network.

To mitigate this risk while ensuring organizational scalability, the components are organized in hierarchical trust models in practice. The following chapter is devoted to this architecture. It analyzes how security is established through a multi-level layered model, how trust moves through a *chain of trust* (CoT), and the algorithmic steps by which a verifying entity validates this path up to the trust anchor.

A. Trust Hierarchies in PKI

The hierarchical trust model represents the de facto standard for establishing scalable trust between unknown parties on the Internet. Instead of requiring a direct trust relationship between all participants, this model abstracts trust through the concept of the Chain of Trust (CoT). In this process, the trust of the top-level authority is transitively transferred to its subordinate authorities

The trust process in a PKI aims to ensure the authenticity of public keys, particularly to prevent man-in-the-middle (MitM) attacks. The first fundamental step is establishing trust in the CA itself. Since a cryptographic chain of trust is not possible without a secure starting point, the root certificate (Root CA) must first be securely distributed to the systems of the verifying entities as a trust anchor. Prominent examples of this are operating systems or web browsers with pre installed Root CAs, since the installation of unaltered original software constitutes an authenticated channel [4, p. 395].

In this context, the globally distributed trust in the private key of the Root CA simultaneously poses the greatest risk. This is because compromising this key jeopardizes trust in all certificates issued by it. To minimize this risk, the [8] established a three-layered model. The three-layer model assigns three distinct functions to the issued CAs

1) **Root-CA**: It forms the foundation of the chain of trust. Its private key is stored in an HSM under the highest security measures, and it operates almost exclusively offline, being activated only during strictly regulated ceremonies to sign certificates at lower levels.
2) **Policy CA** (optional): It forms a strategic isolation layer between Issuing and Root CAs. Typically, Issuing CAs are certified at this level for specific application areas.
3) **Issuing CA**: As the lowest level, it issues certificates to participants.

The concept of the CoT arises directly from this hierarchy. Since an Issuing CA or a participant, when viewed in isolation, is not trusted, this trust must be mathematically derived from the higher-level entities.

Technically, trust in the subscriber is verified through an unbroken chain of signatures leading back to a trust anchor. This chain is constructed using a certificate path. As mentioned in [8], a certificate path corresponds to an ordered list $P = (c_1, c_2, \ldots, c_n)$, for which the following validation rules apply:

1) Initial Condition: The first certificate c_1 was signed by a trust anchor.
2) Path validation: For every pair (c_i, c_{i+1}) where $1 \leq i < n$ the following holds:
   - c_i is the issuer of c_{i+1}.
   - The public key in c_i verifies the signature of c_{i+1}.
3) Target condition: The last element c_n is the participant's certificate to be verified.

Figure 2 illustrates this definition. It shows the certificate chain for Bochum University of Applied Sciences. Furthermore, it demonstrates that the certificate for Bochum University of Applied Sciences (c_2) is not directly trusted by the root CA, but

rather that the chain of trust is validated transitively via c_1 as long as each step $c_i \rightarrow c_{i+1}$ can be cryptographically verified, the end certificate is considered trustworthy. An important note

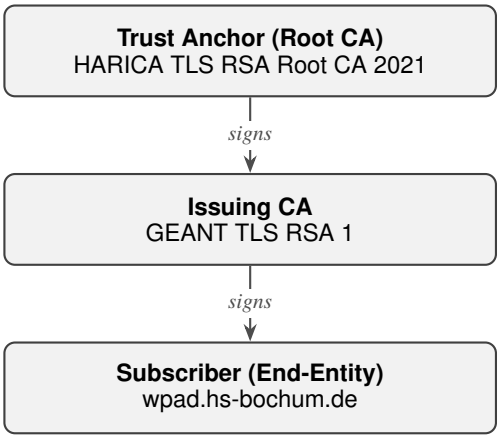

Figure 2: Certificate path $P = (c_1, c_2)$ using the example of the subscriber "Hochschule Bochum". Trust flows from top to bottom.

here is that, according to the Internet Engineering Task Force [8], the trust anchor is not an element of the list itself, but is passed as input to the validation function. The validation function itself is the mapping used to verify the key binding. This is achieved by tracing the certificate holder's binding back to the public key of the trust anchor. To comply with the standard the scope of definition is extended to include a context parameter $\mathcal{C}$ which bundles time and other restrictions:

$$\mathrm{Val} : \mathrm{TrustAnchor} \times P \times \mathcal{C} \rightarrow \{\mathrm{Valid}, \mathrm{Invalid}\}$$

The input parameters are defined as follows:

- **TrustAnchor:** Information about the trust anchor (public key, signature algorithm, and DN).
- **Path (P):** The certificate chain to be verified $(c_1, \ldots, c_n)$.
- **Context ($\mathcal{C}$):** A tuple (t_{now}, Ω), where t denotes the current evaluation time. The set Ω contains policy-related requirements.

This validation function is applied immediately upon connection establishment construction (e. g. in the TLS handshake). Since the verifying authority initially only has its local trust anchor (TA) it is the participant's responsibility to transmit the certificate P including the issuing CA and all intermediate CAs to the validating authority.

The validation process follows the logic of the "Basic Path Validation Algorithm" as defined by [8, Section 6.1]. This algorithm iterates through the certificate path $(c_1, \ldots, c_n)$ and performs a series of checks for each certificate c_i.

The core algorithm can be divided into three logical phases:

1) Initialization: The state variables are initialized with the parameters of the trust anchor (TA). The "working public key" initially corresponds to the root CA's key.
2) Iterative verification ($i = 1 \ldots n$): For each certificate in the chain, the following security conditions are verified:
   - **Signature validation** The digital signature from c_i must be verifiable using the current working public key (from c_{i-1} or TA).
   - **Validity interval:** The current timestamp t_{now} must fall within the period $[\mathit{NotBefore}, \mathit{NotAfter}]$.
   - **Revocation status:** A check is performed to determine whether the certificate has been marked as revoked on a CRL or via OCSP.
   - **Name chaining:** The Issuer-Distinguished-Name of c_i must be identical to the Subject Name of the issuer

   After a successful check, the state variables are updated in the Prepare step: The public key of c_i is extracted and serves as the working public key for the subsequent certificate c_{i+1}.
3) Wrap-Up: After the last certificate c_n, the algorithm terminates. If no step contained an error, the chain is considered cryptographically valid.

An important point to note here is that this algorithm only verifies the cryptographic validity of the chain. It does not verify whether the certificate was actually issued for the requested server. As a result, a certificate may be mathematically valid but still have been issued for the wrong domain.

This discrepancy between mathematical correctness and actual trustworthiness illustrates that the security of a PKI does not rely solely on cryptographic primitives. Rather, the complexity of the processes and the multitude of involved entities create a broad attack surface. The following chapter therefore moves beyond the theoretical examination of how the system works and focuses on the systematic analysis of attack vectors that specifically exploit these vulnerabilities.

IV. Threat Analysis of Hierarchical PKIs

After illustrating the theoretical structure and functionality of a hierarchical PKI in the previous section, this section subjects the trust model to a security analysis. The focus is on pointing out the attack vectors and vulnerabilities that can undermine trust in a hierarchical PKI. This analysis provides the fundamental understanding of the system's volatility that is necessary to subsequently develop effective risk mitigation measures

A. Systematic Threat Classification

To systematically assess the security risks of a hierarchical PKI, the first step is to classify the relevant assets and define the attack vectors to be analyzed

The analysis focuses on cryptographic assets, particularly the private key of the root CA, the compromise of which can lead to a complete loss of trust. However, the private keys of intermediate and issuing CAs are also significant, as this is another channel that attackers can use to issue forged

certificates. Secondary assets include the repositories as well as the communication channels between the RA and the CA.

The classification is based on an attack-oriented perspective. Since an attacker has a variety of attack vectors at their disposal, the simplified representation in the form of a threat tree. Figure 3 consolidates the aforementioned vulnerabilities of the illustrated PKI components section II and classifies them based on their potential impact. The impacts of the respective attack vectors are then evaluated in light of the CIA triad. The analysis is based on the example of the affected company DigiNotar, which fell victim to an external attack.

The goal of this section is therefore not merely to categorize vulnerabilities. Rather, it aims to foster a deep understanding of the fragility of the trust model in order to provide a logical basis for the necessity of subsequent hardening measures.

B. The Cascade of Compromise: Weakest-Link Principle

Based on the analysis by Fox-IT [12], the example of DigiNotar illustrates that the effects of an attack on any node of the threat tree Figure 3 can cascade into a loss of trust in the certification authority:

First, the attackers compromised the root CA and obtained its private key. This allowed them to bypass the RA's identity verification process by issuing unauthorized certificates. Due to the blacklist logic implemented earlier, the revocation mechanisms (CRL/OCSP) also failed, causing the threat to persist for weeks.

While the compromise of a private key represents the most critical security incident, a CA that issues certificates to identities that have not been sufficiently verified or fails to reliably revoke certificates that have been misused is equally untrustworthy. This shows that any node in 3 can lead to a loss of trust in the CA. Accordingly, the system is only as secure as its weakest link (weakest-link principle).

Similarly, user-centric browsers or operating systems, as shown by the Common CA Database [13], trust multiple trust anchors simultaneously. Consequently, a security incident at a certification authority can endanger virtually every end user.

Taking both points into account, it appears necessary to consider hardening measures comprehensively in order to minimize potential risks. Therefore, the following section examines both technical characteristics as well as procedural and other organizational measures that can enhance the security of the ecosystem

V. Contemporary PKI Security and Hardening Strategies

The integrity of a Public Key Infrastructure is not based solely on cryptographic strength but requires the synergy of validation processes and strict operational procedures. To answer the question of how threats can be classified and their risk potential minimized, this section presents measures ranging from real-time status checking via OCSP stapling to transparent certificate issuance.

A. Optimizing (Real-Time) Revocation Checking

The fundamental architectural goal of a PKI is the scalability of trust. With the help of verification algorithms (subsection III-A), any verifying entity can independently verify the integrity of a certificate. However, this process proves impractical when certificates must be revoked before the end of their regular validity period due to security incidents. Inconsistent revocation, as pointed out in section IV, leads verifying entities to classify an invalid certificate as trustworthy.

CRLs and OCSP have historically been the de facto standards for certificate revocation. In practice, however, they pose significant security, privacy, and operational risks. This chapter illustrates these risks and evaluates modern mitigation strategies. In particular, the focus will be on OCSP stapling - both as a conceptual mechanism to enhance privacy and as a highly relevant case study of how operational fragility (such as the deprecation of OCSP- Must-Staple [14]) limits theoretical security models in the modern Web PKI ecosystem.

A significant data protection and performance issue with OCSP is the necessity for the client to communicate with the CA's OCSP responder. As described in [15, p. 60] certification authorities can create behavioral profiles of clients by logging these requests.

defines OCSP stapling as a solution. With OCSP stapling, the certificate holder is responsible for periodically querying the revocation status in the form of a signed response. The participant stores this response in the cache for a limited time (in the case shown in Figure 4.1a, the response is stored for 7 days) and transmits it to the client when establishing a connection via a TLS handshake. While Amazon.de (Listing 1) provides a signed OCSP response including the status "good" directly in the handshake, this information is missing in the connection to Bochum University of Applied Sciences (Listing 2). In the second case, the server forces the client to establish a separate connection to the OCSP responder, which results in the latency and data protection issues described.

```
$ openssl s_client -connect amazon.de:443 -status
[...]
OCSP response:
======================================
OCSP Response Status: successful (0x0)
Cert Status: good
```

Listing 1: successful OCSP-Stapling (Amazon.de)

```
$ openssl s_client -connect hochschule-bochum.de:443
    -status
[...]
OCSP response: no response sent
```

Listing 2: Missing OCSP-Stapling (Hochschule Bochum)

Despite these advantages, there are operational security flaws associated with this solution. One issue is the handling of

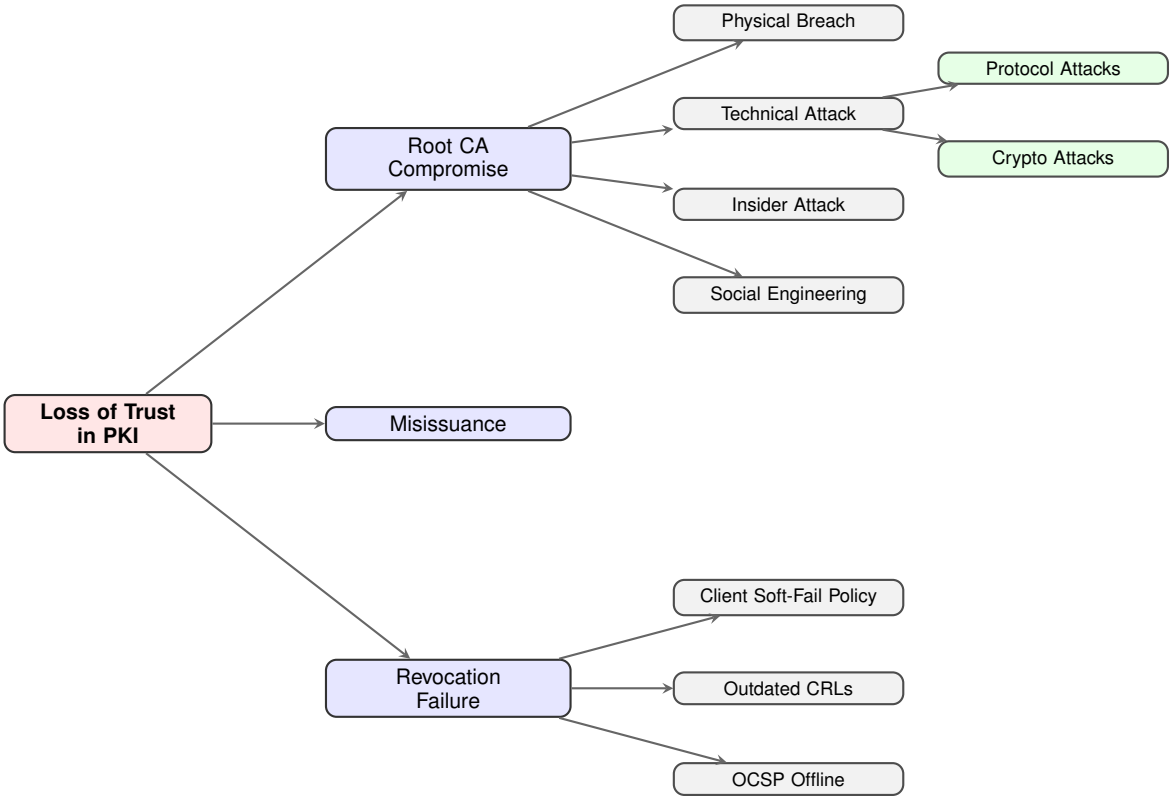

Figure 3: Systematic threat tree classifying attack vectors leading to a total loss of trust.

replay attacks. Since the stapled OCSP response is cached for a specific period of time, attackers could store a response and reuse it within that specific timeframe, even though a certificate has been revoked [16]. Although OCSP does support an operation mode that is not vulnerable to replay attacks, it is rarely supported or used [16].

Furthermore, the overall security benefit of OCSP is severely undercut by the 'soft-fail' policy implemented by major web browsers. To prevent connectivity issues, browsers typically ignore OCSP network errors. Consequently, an attacker can simply block the revocation check, completely bypassing the protection mechanism. While the 'OCSP Must-Staple' extension was introduced to enforce a 'hard-fail', it failed to gain traction due to inconsistent implementation across web servers and a lack of support from leading browser vendors [16].

This failure illustrates the Weakest-Link Principle within the Web PKI ecosystem. The operational implementation of revocation validation is the weakest link. An attack does not need to break the cryptography of the certificate; they merely need to exploit this fragility at the client level thereby rendering the theoretical security model practically ineffective.

While OCSP stapling reduces wait times, Delta CRLs address the problem of the enormous data volumes associated with traditional CRLs. CRLs store information about all revoked certificates, which can accumulate to cause long wait times during transmission in large PKI systems. Delta CRLs solve this by having CAs additionally publish lists that contain only the changes (new revocations) relative to the last complete base CRL. Technically, this is implemented via the deltaCRLIndicator extension. This significantly reduces the amount of data to be transmitted. However, new participants in a PKI must initially receive the full CRL before they can apply the delta updates [8, p. 62].

In summary, it can be stated that CLRs are the preferred method due to reduced implementation complexity. Although both methods optimize transmission, they do not prevent compromised CAs from issuing valid certificates. For this reason, the following section presents a concept that minimizes unconditional trust in the PKI and instead offers a mathematically verifiable procedure.

B. Mitigating Rogue Certificates using Certificate Transparency

Another security standard that PKI systems can utilize is so-called Certificate Transparency (CT). The goal of this security standard is to provide a publicly viewable logging system to make the erroneous and malicious issuance of certificates visible [17, p. 3]. The technical foundation for this mechanism is a cryptographically secured database that operates according to the append-only principle. Specifically, this means that only the addition of new records is permitted, but existing records can never be deleted or modified. This is implemented using a so-called "Merkle tree." A Merkle tree is a special type of

tree constructed from the bottom up using hashes [17, Chapter 2.1]. Referring to the architecture illustrated in [17, Chapter 2.1], the Merkle tree is defined as follows:

Let $\mathcal{D}$ be the set of all possible certificates and $H : \{0,1\}^* \to \{0,1\}^{256}$ be the cryptographic hash function SHA-256. Furthermore let $D_n = (d_0, d_1, \ldots, d_{n-1})$ be an ordered list of n certificates from $\mathcal{D}$.
The *Merkle Tree Hash* (MTH), u and thus the structure of the tree, is defined recursively. We denote the set of leaves by B and the internal nodes by K.

1) Leafs: For a list of length $n = 1$ (a single certificate d_0)the hash value of the leaf B_0 is defined as:

$$MTH(\{d_0\}) = H(\texttt{0x00} \circ d_0) \tag{4}$$

Here, the prefix byte `0x00` serves as a domain separator to protect the second preimage resistance, and $\circ$ m denotes the concatenation of the data.
2) Internal nodes and root: For $n > 1$ let k be the largest power of two that is less than n (i./,.e $k < n \leq 2k$). The list D_n is split at the position k into two sublists $L = (d_0, \ldots, d_{k-1})$ and $R = (d_k, \ldots, d_{n-1})$. The hash of an internal node is calculated from the concatenation of the hashes of its children:

$$MTH(D_n) = H(\texttt{0x01} \circ MTH(L) \circ MTH(R)) \tag{5}$$

Furthermore, relying parties should be able to verify the existence of any certificate d_m in the MTH without having to disclose the entire tree. For this reason, the Merkle audit path method is used. An audit path is the minimal list of all sibling nodes required to recompute the MTH starting from d_m [17, p. 5]. Mathematically, the audit path is calculated as follows:

Let D_n again be the list of certificates and m the index of the certificate $0 \leq m \leq n$ which the audit path is to be calculated. Then the audit path $P(m, D_n)$ is an ordered sequence of hash values defined in [17] as follows:

1) Base case ($n = 1$): The tree consists only of the leaf itself. The path is empty:

$$P(0, \{d_0\}) = \emptyset \tag{6}$$

2) Recursion step ($n > 1$): Let k be the largest power of two such that $k < n$. Let $L = (d_0, \ldots, d_{k-1})$ and $R = (d_k, \ldots, d_{n-1})$ be the subtrees.
   - Case $m < k$ (target in the left subtree): The path is formed in the left subtree. The hash of the right subtree $MTH(R)$ is appended as a necessary sibling node:

$$P(m, D_n) = P(m, L) \circ (MTH(R)) \tag{7}$$

   - Case $m \geq k$ (target in the right subtree): The path is constructed in the right subtree for the local index $m - k$. The hash of the left subtree $MTH(L)$ is appended:

$$P(m, D_n) = P(m - k, R) \circ (MTH(L)) \tag{8}$$

Note: $\circ$ denotes the concatenation of the lists.

The technical advantage of an audit path is its scalability and efficiency. CT logs typically contain millions of entries; therefore, transferring the entire log would result in long wait times for auditing entities. By using the audit path, the verification effort is reduced to logarithmic $O(log_2 n)$ [18, Section 2.1]. Thus, it is also possible for resource-constrained clients to verify whether a certificate is part of the protocol.

Finally, the append-only property of the CT log remains to be proven. Formally, this means that a Merkle tree from a previous version is fully contained within the current version. Let D_n be an ordered list of inputs, and let D_m be an earlier version of length m such that $m \leq n$. A "consistency proof" [17, Section 2.1.2] provides the minimum set of intermediate nodes necessary to verify that D_m is a prefix of D_n and that both generate the respective root hashes $MTH(D_m)$ and $MTH(D_n)$, respectively. The consistency proof function is defined as $\mathcal{C}(m, D_n)$ using a helper function $\mathcal{S}(m, D_n, b)$, where $b \in \{\text{true}, \text{false}\}$ indicates whether this is the initial call for the subtree in question. Let k be the largest power of two less than n. The recursive definition according to [17] is:

$$\mathcal{C}(m, D_n) = \mathcal{S}(m, D_n, \text{true}) \tag{9}$$

$\mathcal{S}$ is defined as:

$$\mathcal{S}(m, D_n, b) = \begin{cases} \emptyset & \text{if } m = n, b = \text{true} \\ \{\text{MTH}(D_n)\} & \text{if } m = n, b = \text{false} \\ \mathcal{S}(m, D_{0:k}, b) \circ \{\text{MTH}(D_{k:n})\} & \text{if } m \leq k, m < n \\ \mathcal{S}(m - k, D_{k:n}, \text{false}) \circ \{\text{MTH}(D_{0:k})\} & \text{if } m > k, m < n \end{cases} \tag{10}$$

where $D_{i:j}$ denotes the subset $\{d_i, \ldots, d_{j-1}\}$.

Since the two-step proof reconstructs both $MTH(D_n)$ and $MTH(D_m)$, the success of this validation proves that D_m is a prefix of D_n . Any subsequent modification of existing data records would, due to the collision resistance of H via the avalanche effect, inevitably lead to a discrepancy in the MTH. To integrate the Merkle tree into the Web PKI, three key roles are defined in [17]:

- Monitor: These instances continuously monitor the logs for suspicious or improperly issued certificates. In doing so, they use the consistency proof to verify that the log maintains its append-only property and that no existing entries have been tampered with retroactively [17, p. 22]
- Auditors: An auditor uses the Merkle Audit Path to prove the existence of a certificate in the log. Since the roles exchange information about the logs' root hashes with one another ("gossip"), this prevents a log operator from presenting different versions to different users [17, p. 23].

- TLS Clients: When establishing a connection, the browser checks whether the certificate presented by the server contains a valid Signed Certificate Timestamp (SCT). An SCT is the log operator's promise that a certificate is contained in the Merkle tree [17, p. 9].

This interaction ensures that the mathematical guarantees of Merkle trees lead to de facto oversight of the certification authorities.

In summary, CT transforms trust in certification authorities from unconditional trust into a mathematically verifiable system, thereby minimizing the damage caused by fraudulent issuance and therefore reducing the impact of the attack vector *Misissuance* (see Figure 3).

C. Security Policies and Operational Resilience

Both organizational and operational guidelines are of particular importance for the tamper-resistant operation of a PKI. For example, attack vectors such as insufficient verification, social engineering, insider attacks, and physical breaches Figure 3 can be minimized through enhanced security standards in these areas.

According to the recommendations in the guideline [6], the integrity of the certification process is a critical link in the chain of trust. A key requirement here is the comprehensive verification of the participant's identity and the hardening of registration processes against attacks. For instance, registration data manipulated by an attacker or outdated data can lead to forged certificates [6, p. 21]. In addition, the cryptographic methods used should always be state-of-the-art [6, p. 35]. subsection V-D provides a deeper insight into the generation of secure key material.

Further process-related organizational measures therefore include the explicit definition and verification of every manual and automated step. Of particular importance here is the clear assignment of responsibilities and roles within the processes. To prevent undetected security vulnerabilities or the inadvertent circumvention of control mechanisms, a strict separation of duties must be ensured [6, p. 56]. This ensures that critical operations, such as the approval of a certificate application, cannot be performed by a single individual without cross-checking by a second authorized party (four-eyes principle). In line with this, the BSI requires the implementation of audit-proof audit logs, which must also be regularly checked for anomalies by dedicated personnel in order to detect security incidents in a timely manner [6, p. 56].

In addition to integrity, the continuous availability of services is also crucial to the security of a PKI. If the CRL or the OCSP responder is unavailable, certificate validation is prevented. Technically, this necessitates the redundant implementation of critical services and a system design capable of consistently handling the expected load. Key parameters include the number of participants, the key lifetime subsection V-D, and the computational effort of a cryptographic operation per participant [6, Chapter 6.13]. Disaster scenarios should also be taken into account, which is why the concepts must be supplemented with disaster recovery measures

Finally, we will consider the planned decommissioning of the PKI. A non-standardized procedure for decommissioning the system jeopardizes the security level, as orphaned cryptographic information could become accessible to attackers. To rule out threats such as the theft of CA keys, all private keys, their copies, and sensitive customer data must be verifiably and irreversibly destroyed [6, p. 75].

In addition, termination requires organizational arrangements, particularly the timely notification of participants and the handover of operations to a trustworthy successor who will assume responsibility for providing revocation lists for the remainder of the active certificates' validity periods. The BSI recommends incorporating the handling of this process and the coverage of remaining operating costs into the terms and conditions from the outset [6, p. 75].

D. Risk Mitigation and Optimal Key Lifecycle Management

As the previous section has already illustrated, the security of a PKI cannot be reduced to the strength of the mathematical algorithms. In addition to organizational requirements, the management of key material also plays a decisive role. This chapter aims in particular to reduce the probabilities and consequences of cryptographic attacks (see 3)

In general, key material should only be used within a predefined time period. This period is known as the cryptoperiod. Within this cryptoperiod, authorized entities are permitted to use the key material. However, cryptoperiods may vary depending on the intended use. An optimal cryptoperiod, as addressed by the National Institute of Standards and Technology [5, p. 34], mitigates cryptanalytic risks, the security benefits of which can be divided into three categories:

1) Prevention
   - The shorter a key is in use, the less ciphertext is available to an attacker to mathematically derive the key.
   - Computationally intensive attacks take time. If the key's lifetime is shorter than the computational time required for an attack, the attack is futile.
   - Controlled key lifetimes prevent cryptographic methods from being used beyond their effective lifespan.
2) Damage Control
   - If a key is compromised, the damage is limited to the period during which the data or communication took place
3) Procedural security
   - Attackers are under time pressure. Short lifetimes make complex attacks to overcome procedural hurdles (e.g., social engineering attacks) unattractive.

Furthermore, the key lifetime of a compromised key should be immediately invalidated

However, determining the optimal key lifetime is a complex challenge, as this determination process depends on various influencing factors (listed in [5, p. 35]). These influencing factors can be categorized as follows:

1) Cryptographic robustness: Strength of the selected algorithms, key length, and threat landscape posed by future technologies
2) Operating environment and transaction volume: The security of the storage environment and the volume of processed operations
3) In particular, the key renewal process plays a role here. Manual processes carry risks in the configuration process. Automated processes (e. g., ACME) thus offer the possibility of using shorter key lifetimes

"**Sensitivity**" [5, Section 5.3.2] is a key factor in determining the key lifetime. This is a metric that takes into account both the lifespan of the information to be protected and the risks associated with a compromise. As a general rule, shorter key lifetimes are selected for information with particularly high sensitivity.

Another essential principle of risk minimization is the disjointed handling of key material intended for different purposes. Furthermore, each key should be used for exactly one process. The following reasons mentioned in [5, p. 33] for this are:

1) The use of keys in different processes weakens the security of at least one of these processes.
2) In the event of a compromise, the damage is limited to the processes that depend on this key.
3) Requirements for the lifecycles of private keys can vary and, in some cases, contradict one another.

The last reason, in particular, requires a detailed explanation: Private signature keys should be destroyed immediately after their key lifetime expires to eliminate the risk of retroactive forgery. In contrast, the long-term archiving of keys used for decryption is often necessary to ensure the availability of archived, encrypted data.

The final phase in a key's lifecycle is its destruction. If shorter key lifetimes are chosen, rotations must be performed more frequently, which inevitably leads to the accumulation of old keys. This situation necessitates effective key destruction, which "makes it impossible to recover the key in electronic or physical form" [5, Section 8.3.4]. However, a blanket deletion of all information would compromise the traceability of PKI operations. It must therefore be possible to determine whether a key was revoked prematurely due to a security incident. For this reason, [5, Section 8.4] requires the archiving of the associated metadata.

The analysis of these measures has demonstrated that while cryptographic standards can reinforce trust within a PKI, they should ideally be based on decentralized, mathematically verifiable procedures. Furthermore, clearly defined procedural and organizational frameworks are essential for hardening the overall security level.

VI. Discussion

This paper focused on analysing the security of hierarchical PKIs. Motivated by the complex structure of a PKI, the goal of this thesis was to classify attack vectors, evaluate hardening measures, and examine their actual use in operational practice.

The analysis in section IV showed that hierarchical PKIs suffer from the weakest link principle. The compromise of a single CA endangers all clients that have stored the underlying trust anchor in their local trust store. The vectors identified cover procedural as well as technical, physical and organizational aspects.

section IV also demonstrated that while threats can be classified, their effects converge significantly, meaning that an attack on a single vector can lead to a complete loss of trust in a CA. Therefore, risk mitigation measures should be implemented across the board. The measures, as outlined in section V, include, among other things, public transparency for the detection of forged certificates, operational diligence, and performance improvements. These measures can minimize risks in terms of probability of occurrence, scope of damage, and time to detection

VII. Limitations of the Results

the focus of this work on the technical level of Web PKI. Human factors (such as phishing attacks) were taken into account to a limited exten (Section V-C).

VIII. Outlook

The PKI is currently evolving from a monolithic trust anchor to a decentralized ecosystem. The findings point to a future trend:

1) The gradual deprecation of OCSP demonstrates that future security gains cannot be achieved by a single entity. Rather, they necessitate a collaboration between browser vendors, Certificate Authorities, and web server operators.

In summary, it can be said that the hierarchical PKI can only remain tamper-resistant if it is constantly willing to adapt. Modern hardening measures must be continuously implemented to withstand the constant threat posed by the cybercriminal shadow economy.

References

[1] Bitkom Research, *Wirtschaftsschutz 2025*, Präsentation zur Pressekonferenz, Sep. 2025.

[2] Bundesamt für Sicherheit in der Informationstechnik (BSI), "Die lage der it-sicherheit in deutschland 2024," Bundesamt für Sicherheit in der Informationstechnik (BSI), Bonn, Tech. Rep., Oct. 2024.

[3] A. Coladangelo, *Quantum trapdoor functions from classical one-way functions*, Cryptology ePrint Archive, Report 2023/282, 2023. [Online]. Available: https://ia.cr/2023/282

[4] C. Paar and J. Pelzl, *Kryptografie verständlich: Ein Lehrbuch für Studierende und Anwender*. Springer, 2010.

[5] National Institute of Standards and Technology (NIST), "Special publication 800-57: Recommendation for key management," National Institute of Standards and Technology (NIST), Tech. Rep., 2020. DOI: 10.6028/NIST.SP.800-57pt1r5

[6] Bundesamt für Sicherheit in der Informationstechnik (BSI), "Bsi tr-03145-1 secure ca operation," Bundesamt für Sicherheit in der Informationstechnik, Tech. Rep., 2021.

[7] J. Ashbourn, *PKI implementation and infrastructures*, First edition. 2023.

[8] Internet Engineering Task Force (IETF), "Rfc 5280: Internet x.509 public key infrastructure certificate and certificate revocation list (crl) profile," IETF, RFC 5280, 2008. [Online]. Available: https://www.rfc-editor.org/info/rfc5280

[9] National Institute of Standards and Technology (NIST), "Fips pub 140-2: Security requirements for cryptographic modules," National Institute of Standards and Technology (NIST), Gaithersburg, MD, Federal Information Processing Standards Publication 140-2, Apr. 2001, Change Notices as of December 3, 2002. Supersedes FIPS PUB 140-1. [Online]. Available: https://csrc.nist.gov/publications/detail/fips/140/2/final

[10] S. Chokhani, W. Ford, R. Sabett, C. Merrill, and S. Wu, "Rfc 3647: Internet x.509 public key infrastructure certificate policy and certification practices framework," IETF, RFC 3647, Nov. 2003. [Online]. Available: https://www.rfc-editor.org/info/rfc3647

[11] Internet Engineering Task Force (IETF), "Rfc 6960: X.509 internet public key infrastructure online certificate status protocol - ocsp," IETF, RFC 6960, 2013. [Online]. Available: https://www.rfc-editor.org/info/rfc6960

[12] Fox-IT, "Operation black tulip: Report of the investigation into the diginotar certificate authority breach," Fox-IT, Investigation Report, version 1.0, Sep. 2012.

[13] The Linux Foundation. "Common ca database (ccadb)," Accessed: Mar. 27, 2026. [Online]. Available: https://www.ccadb.org/resources/

[14] J. Aas. "Ending OCSP support in 2025," Let's Encrypt, Accessed: Mar. 25, 2026. [Online]. Available: https://letsencrypt.org/2024/12/05/ending-ocsp

[15] Bundesamt für Sicherheit in der Informationstechnik (BSI), "Bsi tr-02103: X.509-zertifikate und zertifizierungspfadvalidierung," Bundesamt für Sicherheit in der Informationstechnik, Tech. Rep., version 1.0, Sep. 2020. [Online]. Available: https://www.bsi.bund.de/DE/Themen/Unternehmen-und-Organisationen/Standards-und-Zertifizierung/Technische-Richtlinien/TR-nach-Thema-sortiert/tr02103/tr-02103.html

[16] I. Ristić. "The slow death of OCSP." Cryptography & Security Newsletter, Feisty Duck, Accessed: Mar. 25, 2026. [Online]. Available: https://www.feistyduck.com/newsletter/issue_121_the_slow_death_of_ocsp

[17] B. Laurie, A. Langley, and E. Kasper, "Rfc 6962: Certificate transparency," IETF, RFC 6962, Jun. 2013. [Online]. Available: https://www.rfc-editor.org/info/rfc6962

[18] M. Kirejczyk, M. Kalka, and L. Logvinov, "Historical and multichain storage proofs," vlayer Labs, Whitepaper, Nov. 2024, vlayer.xyz.